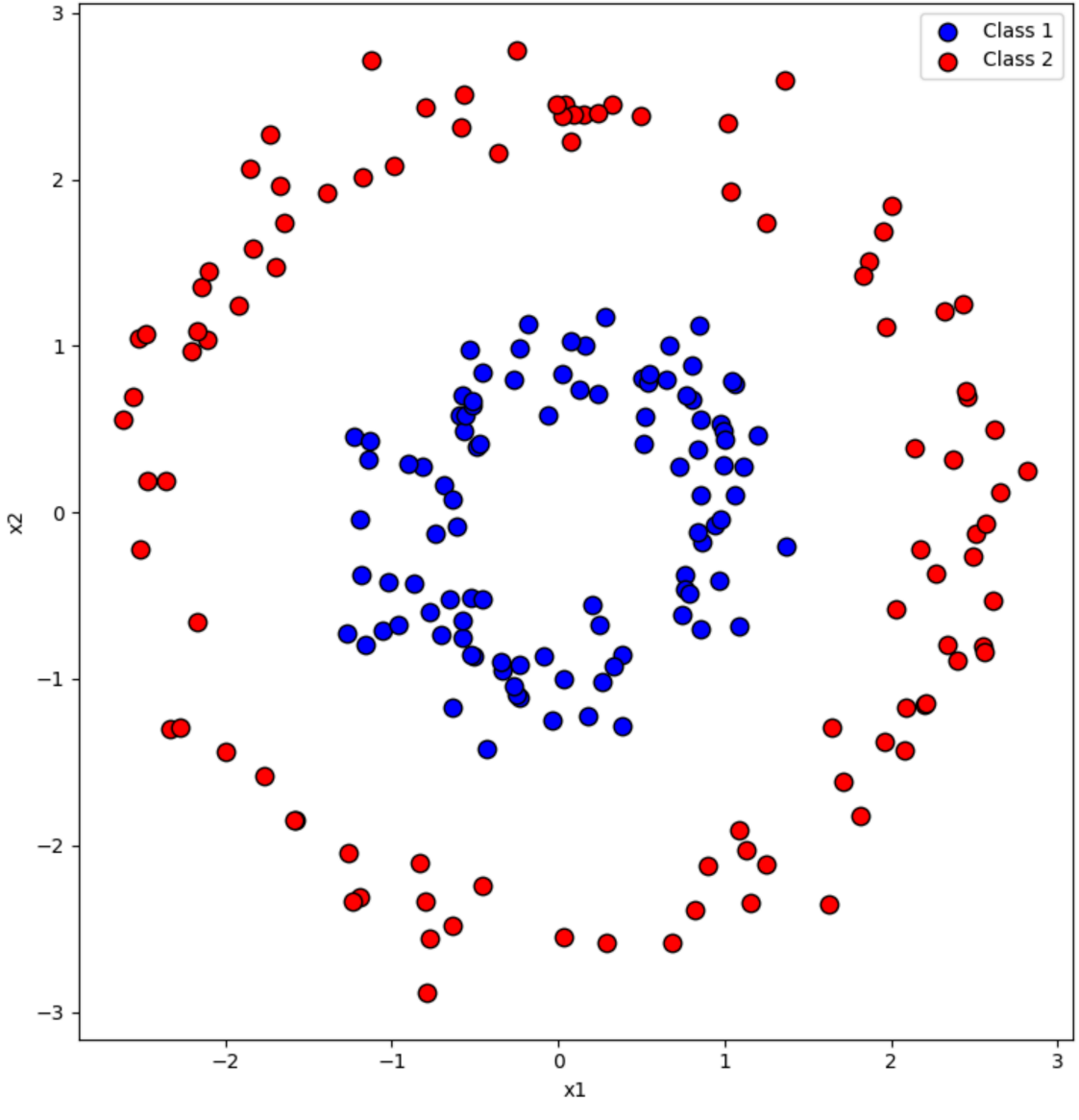
Neural Networks I

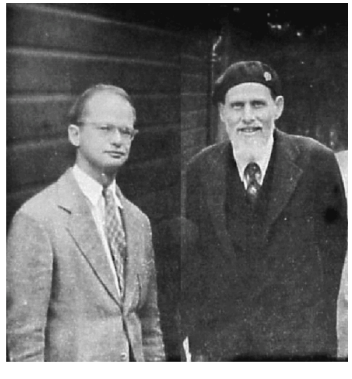
Introduction

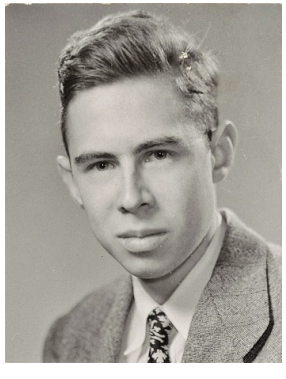
- Our focus has so far been on linear classifiers. Now, our focus will shift to **non-linear classifiers**.
- We will start with learning about neural networks.

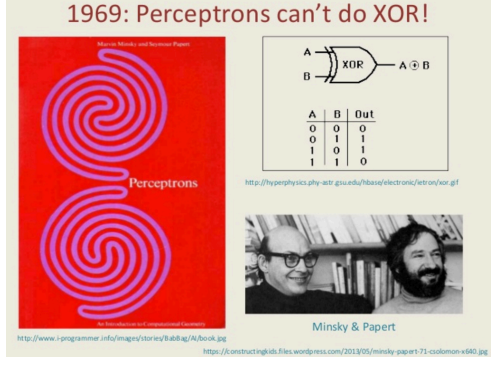

Class 1
Class 2
x1
x2
3
2
1
0
−1
−2
−3
−2
−1
0
1
2
3

Neural networks - a brief history

- McCulloch - Pitts neuron (1946).
- The Perceptron (Rosenblatt, 1950's).
- The XOR problem (Minsky and Papert, 1969) - the first AI winter.
- (Debated topic) Backpropagation (Rumelhart, Hinton, Williams, 1986) - a new hope.
- Lacked data and compute, difficult to train (1990s) - the second AI winter.
- AlexNet (Krizhevsky, Sutskever, Hinton, 2013) - the deep learning revolution.

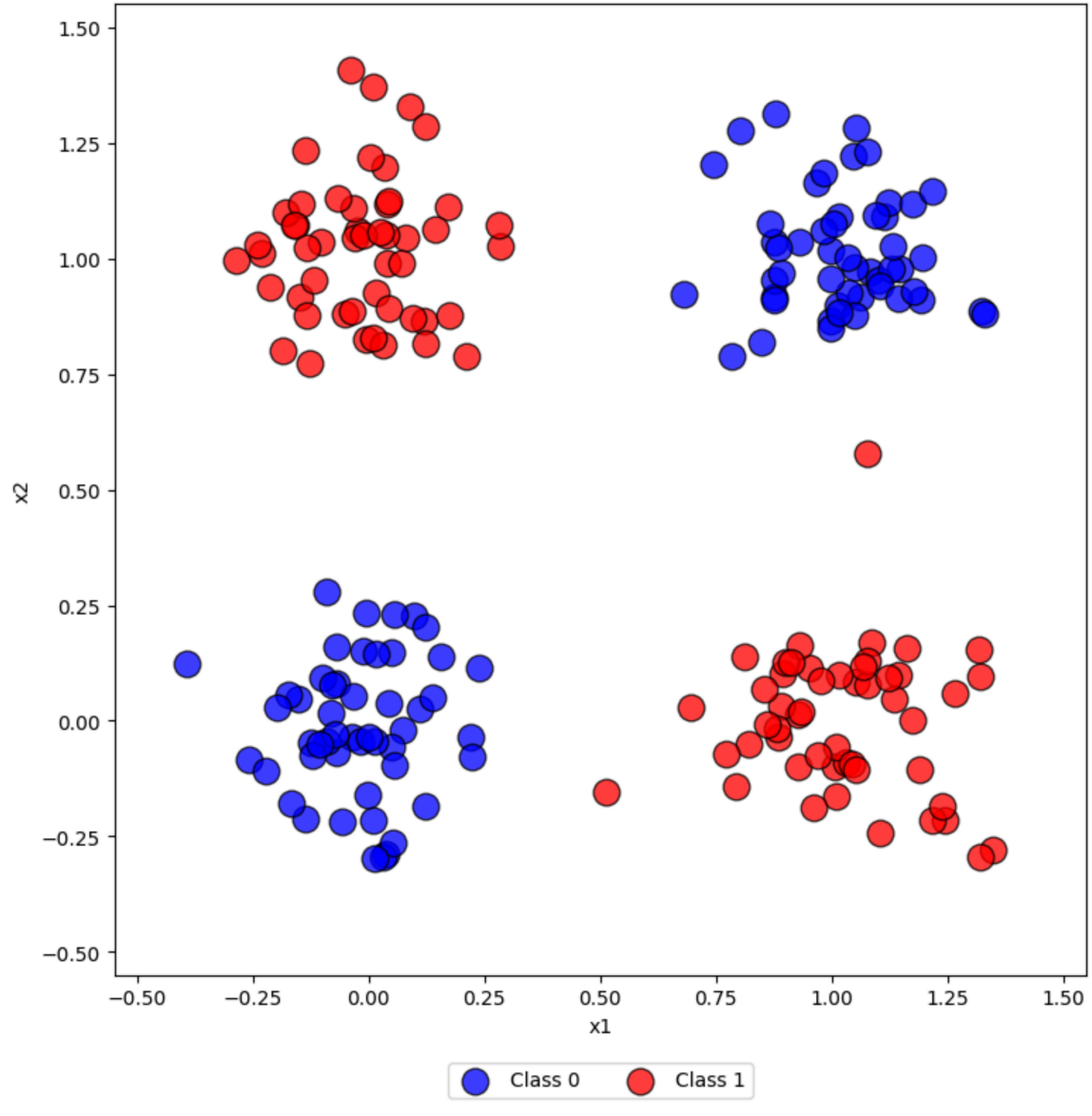






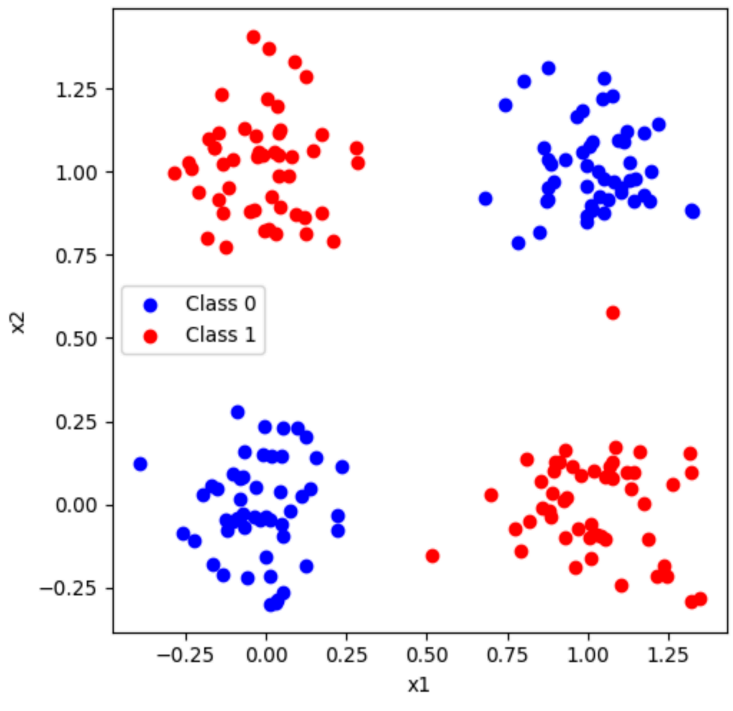
The XOR problem (Minsky and Papert, 1969)

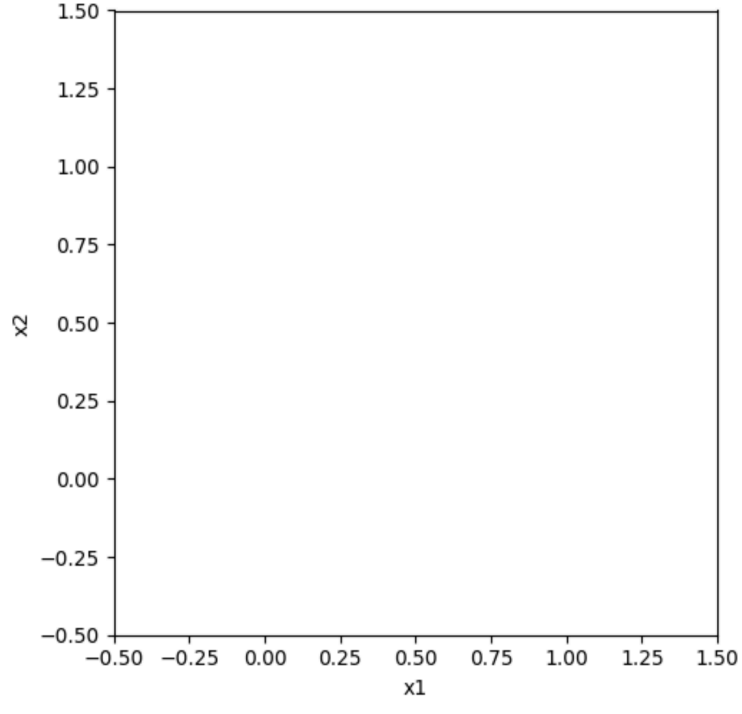
- The Perceptron caused a lot of excitement.
- Researchers started pushing the limit of the Perceptron.
- Problem:
    - The Perceptron cannot solve the XOR problem!


1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
x2
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50
x1
Class 0
Class 1

Transforming into a linearly separable representation

- The perceptron requires linearly separable data.
- What if we could transform the data to accomplish this?


Class 0
Class 1
x1
x2
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.25
0.00
0.25
0.50
0.75
1.00
1.25


x1
x2
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
1.25
1.50

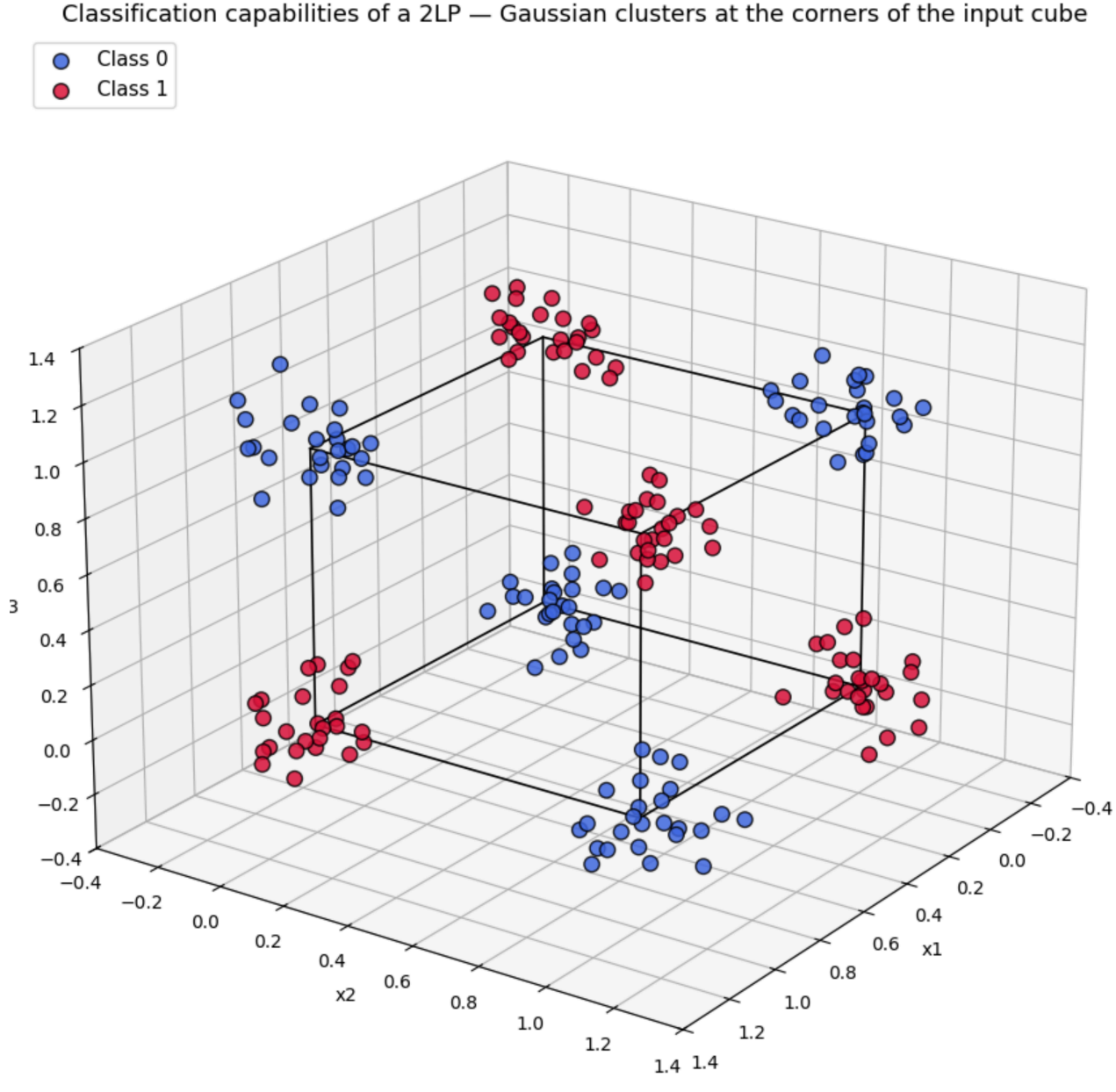
The big idea

- Transform into a new representation where data is linearly separable.
- Transformation is done by another Perceptron!
- Stack layers of Perceptrons to form a **Multilayer Perceptron** (MLP).

A two-layer Perceptron (2LP)

Classification capabilites of 2LP

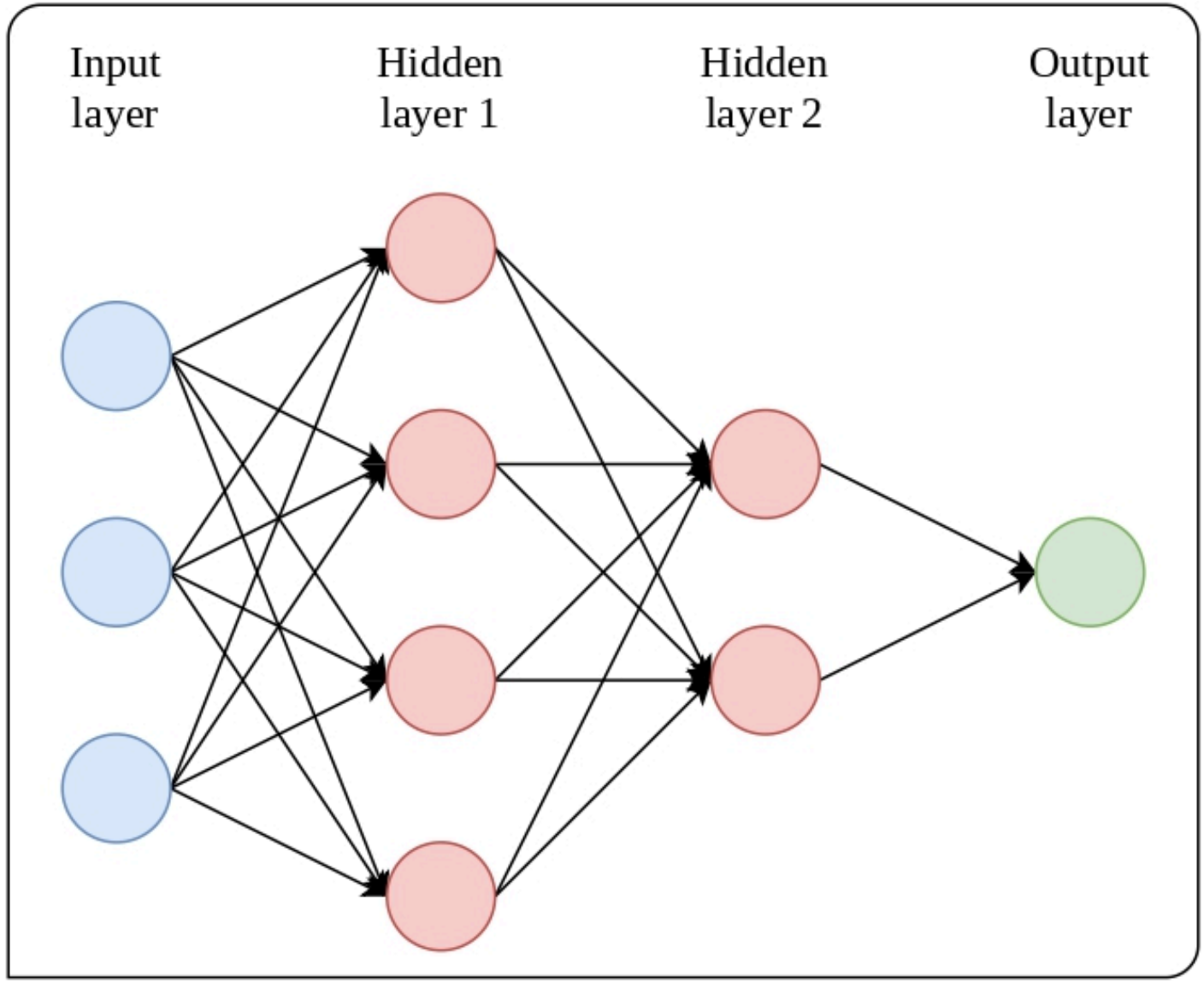
- A hidden layer maps $\mathbf{x} \in \mathcal{R}^d$ onto vertices of a p-dimensional cube (hidden layer with p neurons).
- A 2LP can separate classes represented as union of polyhedral regions.
- Not all regions can be separated!
    - More layers increases the capacity of the network.
    - But how to train?


Classification capabilities of a 2LP — Gaussian clusters at the corners of the input cube
Class 0
Class 1
x1
x2
3

The Backpropagation algorithm

- An efficient training algorithm for MLPs
- Fun blogpost on the topic of who invented Backpropagation.
- Parallel to Stigler's law of eponymy
- A string of important papers:
    - Reverse mode of automatic differentiation (Linnainmaa 1970)
    - Early work on training MLPs with gradient descent (Amari, 1967)
    - Preliminary application of backpropagation in the contex of neural network (Werbos, 1982)
    - Learning representations by back-propagating erros (Rumelhart, 1986)

The Backpropagation algorithm


Input layer
Hidden layer 1
Hidden layer 2
Output layer

The Backpropagation algorithm - loss and optimization

$$J = \sum_{i=1}^{N} E(i),$$

$$E(i) = \sum_{m=1}^{N} \frac{1}{2}\, e_m(i)^2$$

- Want:
  $\min_{\mathbf{w}} J(\mathbf{w})$

The Backpropagation algorithm - gradients of the output layer

$$\frac{\partial E(i)}{\partial \mathbf{w}_j^L} =$$

Gradient descent reminder

$$\mathbf{w}_j^L(t+1) \;=\; \mathbf{w}_j^L(t) \;-\; \gamma \frac{\partial}{\partial \mathbf{w}_j^L} J$$

The Backpropagation algorithm - gradients of the hidden layers

$$\frac{\partial E(i)}{\partial \mathbf{w}_j^{L-1}} =$$

- z_j^{L-1} not present in $E(i)$, comes through a_j^{L-1} and onward to the output.

Gradients of the hidden layers - chain rule

$$\frac{\partial E(i)}{\partial a_j^{L-1}(i)} = \delta_j^{L-1}(i) =$$

$$\frac{\partial a_j^L(i)}{\partial a_j^{L-1}(i)} =$$

$$\delta_j^{L-1}(i) =$$

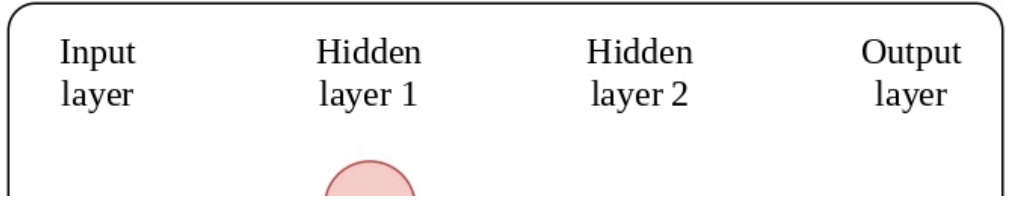
- Error at output (L) propagated to $(L-1)$ to compute gradients of weights.
- Repeat to compute gradients for the entire network!

Backpropagation recipe

- Initialize parameters and "freeze" them.
- The forward pass: compute all
- Backward pass: compute
- Update all parameters
- Repeat!

Top tips for implementing and understanding backpropagation

- Keep it simple in the begining!
- Start with a fixed architecture.
    - Do not have the same number of neurons in a layer.
- Do all calculations by hand before you start programming.
- When you start programming, cross reference your implementation with your hand-calculations.
- There is only one loop, the outer loop of the number of epochs.
    - Matrix multiplication!


Input
layer
Hidden
layer 1
Hidden
layer 2
Output
layer